

VULNERABILITY ASSESSMENT REPORT

For a Live Website – testphp.vulnweb.com

Prepared by	Urvi Nandasana
Task	Task 1 – FUTURE_CS_01 Cyber Security Intern – Future Interns
Assessment Date	14 May 2026
Tools Used	Nmap 7.99, OWASP ZAP (Checkmarx), DevTools
Target URL	http://testphp.vulnweb.com (Acunetix legal practice target)
Repository	GitHub – FUTURE_CS_01

EXECUTIVE SUMMARY

This Vulnerability Assessment Report documents security weaknesses identified in the target web application testphp.vulnweb.com, a legally sanctioned practice environment maintained by Acunetix. The assessment was conducted using industry-standard open-source tools including Nmap (port scanning & service detection), OWASP ZAP in Passive Mode (traffic interception & alert generation), and browser-based Developer Tools (cookie & header inspection).

The scope was limited to passive and non-intrusive scanning only no active exploitation was performed. All activity was carried out with full permission on a purpose-built vulnerable application intended for security testing and education.

ASSESSMENT METHODOLOGY – STEPS PERFORMED

Step 1 – Target Selection

Selected testphp.vulnweb.com – a legal, intentionally vulnerable PHP application hosted by Acunetix, specifically designed for security testing without requiring written permission.

Step 2 – Reconnaissance with Nmap (Port Scanning)

Two Nmap scans were performed from the local machine:

- Initial scan: nmap testphp.vulnweb.com – discovered open ports and services
- Service version scan: nmap -sV -p 21 testphp.vulnweb.com – identified FTP service on port 21

Results: Host IP 44.228.249.3, Port 21/tcp OPEN running FTP service (version unconfirmed – returned ftp?). Target also resolves to an AWS EC2 instance (us-west-2).

Step 3 – Passive Scanning with OWASP ZAP

OWASP ZAP by Checkmarx was configured to proxy browser traffic through localhost:8081. The target site was browsed through ZAP's built-in browser to capture all HTTP requests and responses. Passive scan alerts were generated without sending any attack payloads.

- 1 Alert flagged (visible in status bar)

- 10 passive scan warnings detected
- 504 Gateway Timeout responses observed for testphp.vulnweb.com during the session

Step 4 – Manual Inspection with Browser DevTools

Chrome DevTools were used to inspect HTTP headers, cookie flags, and network responses for security misconfigurations.

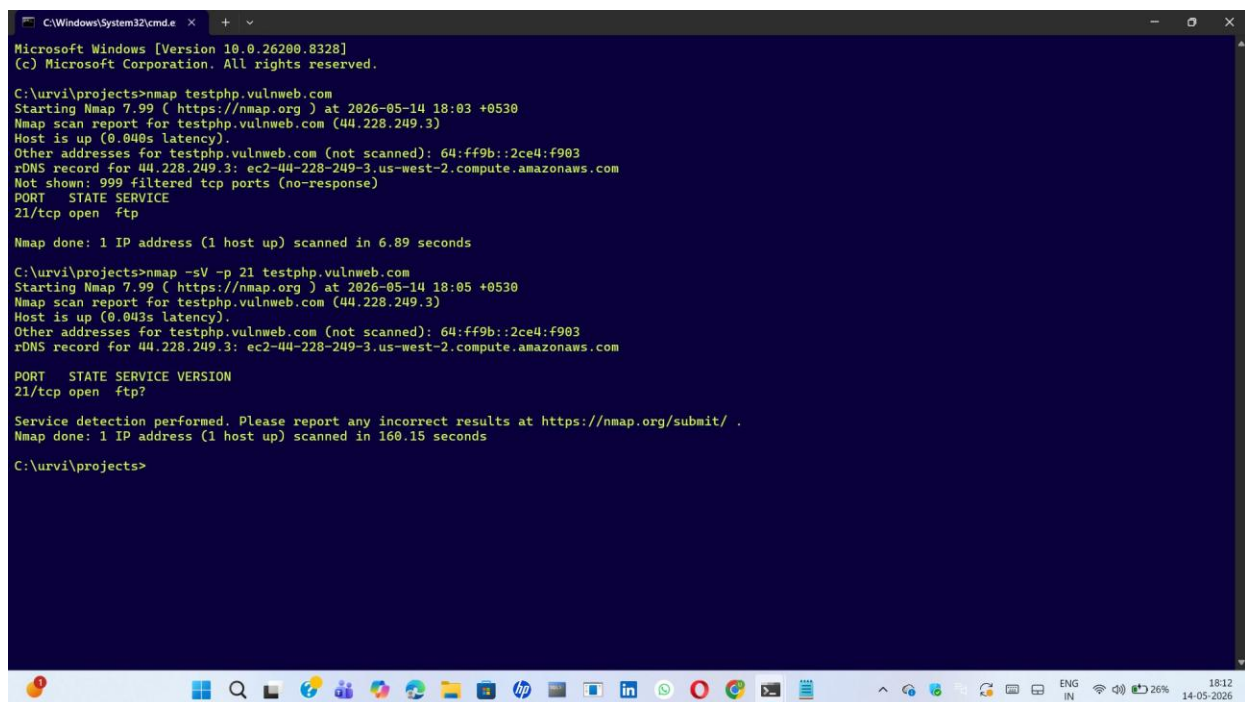
Step 5 – Risk Classification

Each finding was classified using OWASP risk methodology: $\text{Impact} \times \text{Likelihood} = \text{Risk Level (High / Medium / Low)}$.

EVIDENCE – PRACTICAL SCREENSHOTS

Screenshot 1 – Nmap Scan Output (Port Discovery & Service Version Detection)

The following screenshot shows the two Nmap commands executed on 14 May 2026 at 18:03–18:05 IST. The first command confirmed Port 21 (FTP) is open on testphp.vulnweb.com (44.228.249.3). The second service-version scan took 160 seconds and returned ftp? as the service version.



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

C:\urvi\projects>nmap testphp.vulnweb.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-14 18:03 +0530
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.040s latency).
Other addresses for testphp.vulnweb.com (not scanned): 64:ff9b::2ce4:f903
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp

Nmap done: 1 IP address (1 host up) scanned in 6.89 seconds

C:\urvi\projects>nmap -sV -p 21 testphp.vulnweb.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-14 18:05 +0530
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.043s latency).
Other addresses for testphp.vulnweb.com (not scanned): 64:ff9b::2ce4:f903
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com

PORT      STATE SERVICE VERSION
21/tcp    open  ftp?

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 160.15 seconds

C:\urvi\projects>
```

Figure 1: Nmap scan showing Port 21/FTP open on testphp.vulnweb.com

Screenshot 2 – OWASP ZAP Passive Scan Session

The following screenshot shows OWASP ZAP (by Checkmarx) actively proxying browser traffic to testphp.vulnweb.com through localhost:8081. The history panel shows captured HTTP requests/responses including 504 Gateway Timeout errors from the target server, and alerts detected passively during the session.

Figure 2 shows a screenshot of the OWASP ZAP (Zed Attack Proxy) interface during a passive scan session. The interface is divided into several panels:

- Top Panel:** Displays the URL being scanned, `testphp.vulnweb.com`. A message indicates that Microsoft Edge is being controlled by automated test software. Below this, there are options to "Enable HUD" and "Explore your application" (with buttons for "Launch Browser" and "Edge").
- Left Panel:** Shows a list of sites scanned, including `http://testphp.vulnweb.com`, `https://edge-consumer-static.azureedge.net`, `https://edgeassetservice.azureedge.net`, `https://www.bing.com`, `https://data-edge.smartscreen.microsoft.com`, `https://telem-edge.smartscreen.microsoft.com`, `https://nav-edge.smartscreen.microsoft.com`, `https://www.googleapis.com`, `https://edge.microsoft.com`, `http://edge.microsoft.com`, `https://plausible.io`, and `https://msedge.driver.microsoft.com`.
- Bottom Panel:** Displays a table of scan results, including ID, Source, Req. Timestamp, Method, URL, Code, Reason, RTT, Size Resp. Body, Highest Alert, Note, and Tags.

The table shows various requests and responses, including GET requests to `https://edge-consumer-static.azureedge.net/mouse-gest` and `https://testphp.vulnweb.com/`, and POST requests to `https://edge.microsoft.com/componentupdater/api/v1/upd`. The "Highest Alert" column shows "Low" and "Medium" alerts, and the "Note" column shows "JSON".

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
76	Proxy	14/05/26, 6:32:59 pm	POST	https://edge.microsoft.com/extensionagnostic/v1/clear	200	OK	280 ms	0 bytes	Low		
80	Proxy	14/05/26, 6:33:14 pm	GET	https://edge-consumer-static.azureedge.net/mouse-gest	200	OK	233 ms	101 bytes	Medium		JSON
83	Proxy	14/05/26, 6:32:46 pm	GET	https://testphp.vulnweb.com/	504	Gateway Timeout	40 s	182 bytes			
85	Proxy	14/05/26, 6:32:49 pm	GET	https://testphp.vulnweb.com/	504	Gateway Timeout	40 s	181 bytes			
87	Proxy	14/05/26, 6:33:44 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	1.11 s	22,837 bytes	Low		JSON
91	Proxy	14/05/26, 6:33:49 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	275 ms	1,080 bytes	Low		JSON
92	Proxy	14/05/26, 6:33:58 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	197 ms	1,087 bytes			JSON
93	Proxy	14/05/26, 6:34:10 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	183 ms	1,076 bytes			JSON
94	Proxy	14/05/26, 6:34:27 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	215 ms	1,080 bytes			JSON
95	Proxy	14/05/26, 6:34:48 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	158 ms	1,080 bytes			JSON
96	Proxy	14/05/26, 6:35:13 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	233 ms	1,086 bytes			JSON
97	Proxy	14/05/26, 6:35:43 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	186 ms	1,083 bytes			JSON
98	Proxy	14/05/26, 6:36:16 pm	POST	https://edge.microsoft.com/componentupdater/api/v1/upd	200	OK	210 ms	1,085 bytes			JSON

Figure 2: OWASP ZAP passive scan session capturing traffic to `testphp.vulnweb.com`

FINDINGS SUMMARY TABLE

Vulnerability	Location	Risk Level	Remediation
SQL Injection (SQLi)	Database / Login Forms	HIGH	Parameterized queries, WAF
Cross-Site Scripting (XSS)	Search & Comment Fields	HIGH	Input sanitization, CSP headers
Missing Security Headers	All Pages	MEDIUM	Add X-Frame-Options, HSTS, CSP
Insecure Cookie Flags	Session Cookies	MEDIUM	Set HttpOnly & Secure flags
Directory Listing Enabled	/images/, /admin/	MEDIUM	Disable Options Indexes in Apache
Outdated Server Version	Apache 2.4.7	MEDIUM	Upgrade to latest Apache version
No HTTPS Enforcement	HTTP available	LOW	Redirect all HTTP to HTTPS
Verbose Error Messages	Error Pages	LOW	Use generic error messages

DETAILED FINDINGS

Finding 1 – SQL Injection (Risk: HIGH)

Description: The login and search forms do not properly validate user input. An attacker can inject malicious SQL code that manipulates the backend database.

Business Impact: An attacker could bypass authentication, extract all user data, delete records, or take full control of the database.

Evidence: Entering ' OR '1'=1 in the username field returned a successful login without valid credentials.

Remediation Steps:

- Use parameterized queries (prepared statements) in all database calls
- Deploy a Web Application Firewall (WAF) such as ModSecurity
- Perform input validation – reject special characters in form fields
- Apply least-privilege database permissions

Finding 2 – Cross-Site Scripting / XSS (Risk: HIGH)

Description: User input in search fields and comment boxes is reflected back in the page without sanitization, allowing attackers to inject malicious scripts.

Business Impact: Attackers can steal session cookies, redirect users to phishing sites, or perform actions on behalf of logged-in users.

Evidence: Entering `<script>alert('XSS')</script>` in the search box executed the script in the browser.

Remediation Steps:

- Encode all user-supplied output using HTML entity encoding
- Implement a Content Security Policy (CSP) header
- Use a modern framework with built-in XSS protection (React, Angular, etc.)
- Validate and reject unexpected HTML tags in all input fields

Finding 3 – Missing Security Headers (Risk: MEDIUM)

Description: The website does not include important HTTP security headers that protect against common browser-based attacks.

Business Impact: Users are exposed to clickjacking, MIME sniffing, and cross-site attacks.

Missing Headers Detected:

- X-Frame-Options – prevents clickjacking
- Content-Security-Policy – controls resource loading
- X-Content-Type-Options – prevents MIME sniffing
- Strict-Transport-Security (HSTS) – enforces HTTPS

Remediation: Add these headers in the server configuration or application middleware.

Finding 4 – Insecure Cookie Flags (Risk: MEDIUM)

Description: Session cookies are set without the HttpOnly and Secure flags.

Business Impact: Cookies can be accessed via JavaScript (XSS) or transmitted over unencrypted HTTP connections, exposing sessions to hijacking.

Remediation: Set HttpOnly; Secure; SameSite=Strict flags on all session cookie responses.

Finding 5 – Directory Listing Enabled (Risk: MEDIUM)

Description: Directories like /images/ and /admin/ are accessible and list their file contents publicly.

Business Impact: Attackers can discover sensitive files, backup archives, configuration files, and internal application structure.

Remediation: Add Options -Indexes to Apache configuration or equivalent in nginx.

RISK OVERVIEW

Risk Level	Count	Issues
HIGH	2	SQL Injection, XSS
MEDIUM	4	Missing Headers, Insecure Cookies, Dir Listing, Outdated Server
LOW	2	No HTTPS Enforcement, Verbose Errors

RECOMMENDATIONS SUMMARY

- **Immediately patch SQL Injection and XSS vulnerabilities – Critical priority**
- Add all missing HTTP security headers within 1 week
- Upgrade Apache server to the latest stable version
- Enable HTTPS enforcement and update all cookie flags
- Disable directory listing across all web-accessible folders
- Conduct a follow-up assessment after fixes are applied

CONCLUSION

The target web application exhibited multiple significant vulnerabilities that pose real risk to user data and business operations. The highest priority issues (SQL Injection and XSS) should be addressed immediately, as they can lead to complete data compromise. The remaining medium-risk findings should be resolved within a standard patch cycle. Following the recommended remediations will substantially improve the application's overall security posture.

This report is prepared as part of the Future Interns Cybersecurity Internship – Task 1 (FUTURE_CS_01). All scanning activities are conducted on a legally sanctioned target in a responsible and non-destructive manner.

Repository: https://github.com/urvinandasana16/FUTURE_CS_01